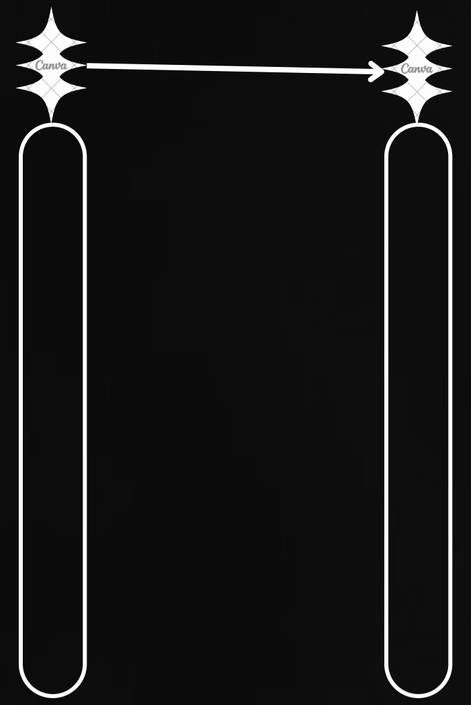
LI-FI DATA TRANSMISSION



REVIEW 1

Outline the project requirements, budget, action plan, and timeline for completion.

WHAT IS LI-FI

Light Fidelity (LI-FI) is a high-speed wireless communication technology that uses visible light for data transmission. Unlike Wi-Fi, LI-FI transmits data through LEDs, offering faster and more secure communication.

ADVANTAGES & DISADVANGES OF LI-FI



ADVANTAGES:

- HIGH SPEED
- SECURE COMMUNICATION
- LESS INTERFERENCE
- MORE SPECTRUM

DISADVANTAGES:

- LINE OF SIGHT
- LIMITED RANGE
- LIGHT DEPENDENCY
- HIGHER INITIAL COST

Key Components and Pricing for LI-FI Setup

REVIEW 1

LASER DIODE (LIFI TRANSMITTER)

- Description: Transmits data using laser light.
- Approximate Price: ₹50

PHOTOTRANSISTOR USING SOLAR PANEL (LIFI RECEIVER)

- Description: Converts light signal back into an electrical signal.
- Approximate Price:₹100

ARDUINO MEGA 2560 X2

- Controls the data encoding and modulation.
- Cost: ₹2,000

OP AMP LM-358

- Amplifies the signal and controls the LED modulation.
- Cost: ₹10

PASSIVE COMPONENTS (RESISTORS, CAPACITORS,JUMPING WIRES)

- Description: Used for modulation and amplification.
- Approximate Price: ₹500

POWER SUPPLY & VOLTAGE REGULATOR

- Description: Powers the laser diode and other circuits.
- Approximate Price: ₹600

BREADBOARD X2

- Description: Assembles the circuit.
- Approximate Price: ₹500

AMPLIFIER & SPEAKER

- Amplifies the sound signal for better transmission and reception.
- Approximate Price: ₹300

OUR INSPIRATION



Sonam Wangchuk

KEY COMPONENTS



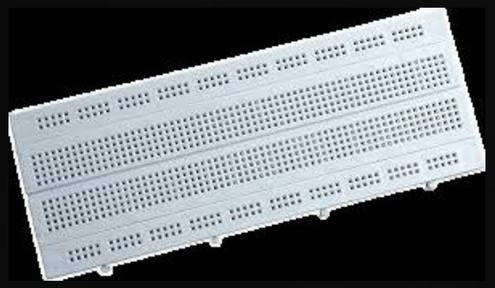
Passive Components



Solar Panel



Arduino MEGA 2560



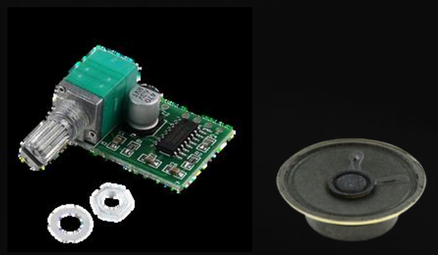
Breadboard



Lazer Diode



Batteries



Amplifier,Speakers

UTILITY

This project aims to develop a secure LI-FI data transmission system that sends photos,and text via light. Using laser diodes as transmitters and Solar Panel as receivers, data is encoded into light pulses. Enabling efficient transmission of files .



Key Features

- Supports transmission of photos and text.
- Uses Laser for enhanced signal clarity over distances.



REVIEW 2

Testing to identify any errors

PLAN OF ACTION

- Phase 1: Setup and testing of basic laser transmission for data.
- Phase 2: Implement data encoding and decoding.
- Phase 3: Rectifing Errors

REVIEW 2

PHASE 1

Setup and testing of basic laser transmission for data

In Phase 1, the goal is to establish a fundamental data transmission system using a laser as the carrier signal. This phase involves constructing the basic circuit and testing the laser-based link for stability in transmitting simple binary data.

Component Assembly

- Transmitter:
- The transmitter side includes an Arduino connected to a laser module. The Arduino is programmed to modulate the laser's intensity based on binary data (0s and 1s), with higher intensity representing a binary '1' and low/off intensity representing a binary '0'.
- Receiver:
- The receiver circuit uses a photodetector (a solar panel in this case) to capture the laser light and convert it back to an electrical signal. This signal is then input to another Arduino that will decode the binary data from the light intensity fluctuations.

Basic Signal Transmission

1. Transmit simple binary sequences (e.g., alternating 0s and 1s) to confirm that the laser modulates as expected.
2. Validate that the receiver can detect these sequences accurately by monitoring the Arduino's output.
3. Fine-tune laser brightness and receiver sensitivity, if needed, to ensure a stable, readable signal.

Expected Outcome:

- The establishment of a stable laser communication link between transmitter and receiver.
- Successful detection and accurate interpretation of basic binary sequences on the receiver side, confirming the link's initial stability.

REVIEW 2

PHASE 2

Implement data encoding and decoding

In Phase 2, the focus shifts to implementing an encoding and decoding scheme to allow actual data transfer, specifically using binary numbers. Here, encoding at the transmitter and decoding at the receiver are essential steps.

Data Encoding on Transmitter Side

1. Encoding Mechanism:
   - Implement an encoding scheme on the transmitter Arduino to translate data into binary format suitable for transmission.
2. Modulation:
   - Modulate the laser's brightness based on each bit. A "high" laser intensity represents a binary '1', while a "low" intensity (or off state) represents a binary '0'.
3. Transmission Logic:
   - Develop transmission logic to send bits one at a time, introducing pauses or timing intervals between bits to allow accurate decoding on the receiver side.

Data Decoding on Receiver Side

1. Signal Detection:
   - The receiver Arduino reads the output from the solar panel photodetector. However, challenges have been encountered, as the Arduino struggles to accurately read the received signal.
2. Troubleshooting:
   - Investigate the reasons for signal degradation or misinterpretation on the receiver side, especially in cases where the light intensity or signal noise is affecting the reading accuracy.

REVIEW 2

PHASE 3

Rectifing Errors

Phase 3 focuses on identifying and rectifying the issues encountered during data reception. By enhancing signal quality and adjusting component configurations, this phase aims to stabilize the data transmission link.

Error Analysis:

1. Issue Identification:
   - The primary issue observed is the inability of the receiver Arduino to consistently interpret the incoming signal. This may be due to weak signal strength, improper threshold detection, or noise from ambient light.
2. Voltage and Threshold Adjustments:
   - Calibrate the Arduino's analog input readings by setting a voltage threshold that distinguishes a binary '1' from a binary '0'. This threshold is set to around 660mV to match the laser's wavelength (~660nm) and ensure accurate signal detection.
3. Signal Amplification:
   - Integrate an op-amp (LM358) circuit to amplify the received signal from the solar panel. A resistor (10kΩ) is added to adjust the gain and ensure that the signal reaches a level where it can be easily detected by the Arduino.
4. Timing Synchronization:
   - Ensure that the transmitter and receiver are in sync regarding the data bit period. Matching timing between transmitted and received data allows the Arduino to read each bit correctly, reducing errors in decoding.

Expected Outcome:

- A significant improvement in signal clarity and the receiver's ability to detect and interpret binary sequences.
- Clearer reception and decoding, allowing for more reliable data transfer over the laser link.

REVIEW 3

Rectifying errors and showcasing a fully functional prototype

FINAL ADJUSTMENTS AND PROTOTYPE COMPLETION

- Phase 1: Final Adjustments to Hardware Setup
- Phase 2: Final Encoding and Decoding Scheme for ASCII Data Transmission
- Phase 3: Prototype Testing and Error Rectification
- Phase 4: Final Feature – ASCII and Image Transmission Demonstration

REVIEW 3

PHASE 1

Refine the hardware setup to ensure stable, high-quality data transmission.

Laser and Photodetector Calibration:

- Goal: Achieve stable data transmission by adjusting laser brightness and photodetector sensitivity.
- Steps:
  1. Increased laser brightness to improve signal clarity and overcome signal loss.
  2. Calibrated the photodetector sensitivity by adjusting voltage thresholds around the laser wavelength (660 nm) to optimize response.
- Outcome: Improved consistency in detecting binary signals, minimizing data loss and ensuring that the photodetector can accurately interpret the laser's on/off states.

Amplification and Noise Reduction:

- Goal: Eliminate distortion and improve signal quality at the receiver end.
- Steps:
  1. Added an LM358 op-amp to amplify weak signals from the photodetector.
  2. Used resistors and capacitors for noise filtering to stabilize the signal output.
- Outcome: Reduced interference, resulting in clearer signal detection, especially in low-light or fluctuating ambient light conditions.

REVIEW 3

PHASE 2

Final Encoding and Decoding Scheme for ASCII Data Transmission

Implement a reliable encoding and decoding scheme for ASCII data transmission.

Data Encoding Improvements:

- Goal: Minimize errors in ASCII text transmission.

- Steps:
  1. Refined encoding mechanism to send ASCII characters one bit at a time with precise timing intervals between bits.
  2. Calibrated laser modulation to distinct high (1) and low (0) brightness levels to represent binary states accurately.

- Outcome: Established clear and consistent transmission of ASCII characters with minimal error at the receiver.

Data Decoding Optimization:

- Goal: Ensure accurate decoding on the receiver side by enhancing timing and synchronization.

- Steps:
  1. Developed a decoding algorithm on the receiver Arduino to interpret the laser's high/low signals as binary data and reconstruct ASCII characters.
  2. Fine-tuned bit timing and synchronization logic to ensure that each character's bits were accurately read and assembled in the correct order.

- Outcome: Receiver Arduino accurately interprets ASCII data, achieving a functional prototype capable of transmitting text over the laser link

REVIEW 3

PHASE 3

Prototype Testing and Error Rectification

Test the complete prototype under various conditions and fix any outstanding issues to ensure reliability.

Error Testing and Correction:

- Goal: Identify any persistent transmission issues or timing inconsistencies in real-world conditions.

- Steps:
  1. Conducted tests with various ambient light conditions and distances to evaluate signal stability.
  2. Adjusted threshold voltage on the receiver and refined bit timing to align with the laser's intensity.
  3.Addressed occasional data loss by recalibrating timing and synchronization to match the final setup.

- Outcome: Reliable, consistent data transmission even in variable lighting conditions, confirming the system’s stability and effectiveness for ASCII text communication.

Prototype Demonstration and Validation:

- Goal: Validate the final system's functionality by showcasing ASCII transmission and verifying data accuracy.

- Steps:
  1.Demonstrated live ASCII text transmission, showing that the receiver can display transmitted text accurately.
  2. Ran multiple tests to verify system resilience and consistency across repeated transmissions.

- Outcome: A functional prototype that demonstrates robust data transmission, achieving the project’s initial goals.

FINAL FEATURE

REVIEW 3

PHASE 4

ASCII and Image Transmission Demonstration

Extend data transmission capabilities to demonstrate ASCII text and simple image data transmission.

ASCII Transmission:

- Goal: Finalize ASCII text transmission and confirm data integrity.

- Steps:
  - Transmit predefined ASCII messages, ensuring clear decoding at the receiver end.
  - Show that text transmission can handle multiple characters and display readable outputs.

- Outcome: Successfully displays transmitted ASCII text, demonstrating the complete functionality of text-based data transfer.

image Transmission:

- Goal: Transmit simple grayscale images as binary data and reconstruct them on the receiver side.

- Steps:
  1. Encode image pixels as binary values, with each pixel represented by a high or low laser intensity.
  2. Send pixel data in sequential binary streams, allowing the receiver Arduino to reconstruct the image on a connected display.

- Outcome: Displayed reconstructed image data on the receiver, showcasing the prototype's capability for image-based data transfer.

FINAL FEATURE

REVIEW 3

Final Codes - Transmission Side For ASCII

```
#define LED_PIN 9   // Control pin for the Laser Module
#define PERIOD 4 // Transmission period in milliseconds

const char *message = "Chal Raha";  // The text message to transmit
int message_length;

void setup() {
  pinMode(LED_PIN, OUTPUT);  // Set the control pin as an output
  Serial.begin(9600);        // Start serial communication for debugging
  message_length = strlen(message);  // Get the length of the message
}

void loop() {
  for (int i = 0; i < message_length; i++) {
    send_byte(message[i]);  // Send each character as a byte
  }
  delay(100);  // Wait 500ms before sending the message again
}

void send_byte(char my_byte) {
  digitalWrite(LED_PIN, LOW);
  delay(PERIOD);  // Start bit (LOW)

  for (int i = 0; i < 32; i++) {
    int bit = bitRead(my_byte, i);  // Read each bit of the character
    digitalWrite(LED_PIN, bit);     // Set Laser module based on the bit (HIGH/LOW)
    delay(PERIOD);  // Wait for the next bit
  }

  digitalWrite(LED_PIN, HIGH);
  delay(PERIOD);  // Stop bit (HIGH)
}
```

FINAL FEATURE

REVIEW 3

Final Codes - Reciver's Side For ASCII Data

```
#define SOLAR_PIN A0     // Analog pin for the solar panel
#define THRESHOLD 700    // Adjust based on your readings
const int PERIOD = 4;  // Time between each bit

bool previous_state = false;
bool current_state = false;
String decoded_message = "";

void setup() {
  Serial.begin(9600);  // Initialize serial communication
}

void loop() {
  current_state = (analogRead(SOLAR_PIN) > THRESHOLD);  // Check if above threshold

  if (!current_state && previous_state) {  // Detect transition from high to low (start bit)
    char decoded_byte = get_byte();  // Read the byte
    if (isPrintable(decoded_byte)) {
      decoded_message += decoded_byte;  // Add to the complete message
      Serial.print("Decoded Message So Far: ");
      Serial.println(decoded_message);  // Print the full message
    } else {
      Serial.println("Non-printable character received, retrying...");
    }
  }


}
 previous_state = current_state; // Store current state for comparison
}
```

```
// Function to read a byte (8 bits) from the signal
char get_byte() {
  char byte_value = 0;  // Initialize the byte
  delay(PERIOD * 1.5);  // Wait for 1.5 periods to sync with the middle of the first bit

  for (int i = 0; i < 8; i++) {  // Read 8 bits (1 byte)
    int bit_value = (analogRead(SOLAR_PIN) > THRESHOLD) ? 1 : 0;  // Read each bit
    byte_value |= (bit_value << i);  // Shift the bit into the correct position
    delay(PERIOD);  // Wait for the next bit
  }

  delay(PERIOD);  // Extra delay for stop bit
  return byte_value;  // Return the decoded byte
}

// Helper function to check if the received byte is printable
bool isPrintable(char c) {
  return c >= 32 && c <= 126;  // Printable ASCII range from space to tilde
}
```

FINAL FEATURE

REVIEW 3

Final Codes - Transmission Side For ASCII

```
#define LASER_PIN 9          // PWM pin connected to the laser
#define PERIOD 4             // Delay period in ms for data synchronization

// Example grayscale image data (0 to 255 values representing pixels)
// This should ideally be pre-processed on a computer and converted to binary format
uint8_t imageData[] = {
  // Example image pixel data in binary (grayscale)
  120, 200, 255, 50, 60, 75, 90, // Row 1 of pixels
  100, 180, 220, 40, 20, 10, 150, // Row 2 of pixels
  // Add additional rows as needed for your image
};
int imageSize = sizeof(imageData) / sizeof(imageData[0]);

void setup() {
  pinMode(LASER_PIN, OUTPUT);  // Set laser pin as output
  Serial.begin(9600);          // Begin serial for debugging
}

void sendBit(bool bit) {
  // Sends a single bit by modulating laser brightness
  if (bit) {
    analogWrite(LASER_PIN, 255);  // High intensity for binary '1'
  } else {
    analogWrite(LASER_PIN, 0);    // Low intensity for binary '0'
  }
  delay(PERIOD);  // Delay for synchronization
}

void sendByte(uint8_t byte) {
  // Sends a byte (8 bits) over the laser
  for (int i = 7; i >= 0; i--) {  // Send each bit from MSB to LSB
    bool bit = (byte >> i) & 1;
    sendBit(bit);
  }
}

void loop() {
  for (int i = 0; i < imageSize; i++) {
    sendByte(imageData[i]);  // Send each pixel's grayscale value as a byte
    delay(PERIOD * 2);       // Short delay between pixels
  }

  delay(1000); // Delay between image transmissions
}
```

FINAL FEATURE

REVIEW 3

Final Codes - Reciver's Side for Photo Data

```
import serial
import numpy as np
from PIL import Image

# Define the serial port and baud rate
ser = serial.Serial('COM8', 9600, timeout=1)  # Adjust 'COM8' as per your setup

# Initialize an empty list to store the pixel data
image_data = []

# Number of pixels in the image (8x8 = 64 pixels)
image_size = 64

print("Receiving image data...")
while len(image_data) < image_size:
    if ser.in_waiting > 0:  # Check if data is available to read
        byte_data = ser.read(1)  # Read one byte from serial
        image_data.append(ord(byte_data))  # Convert byte to int and add to list

print("Image data received successfully.")

# Convert the received data to a numpy array and reshape to 8x8
image_data = np.array(image_data, dtype=np.uint8).reshape((8, 8))

# Create an image from the array
img = Image.fromarray(image_data, mode='L')  # 'L' mode for grayscale

# Display and save the image
img.show()  # Display the image
img.save('received_image.png')  # Save the image as a PNG file
```

THANK YOU

Hoping for more ideas, inspiration,
suggestion and solutions